Verslag labo 4

Opdracht



De opdracht bestaat uit 2 verschillende delen. In deel 1 bouwen we voort op de oplossing van het vorige labo, hierbij moeten we namelijk de rood component van een afbeelding inverteren, dit zowel met gebruik van coalesced en un-coalesced memory access, en het verschil van berekentijd tussen beiden vergelijken. In deel 2 zullen we een kernel schrijven dat 2 matrices zal vermenigvuldigen. Dit met gebruik van global-only, global and shared, en global and constant memory. Ook hier zullen we de berekentijd van de verschillende gebruikte geheugens vergelijken.

Werkwijze

Om deze opdracht te verwezenlijken werd stap per stap gewerkt. Nadat deel 1 afgewerkt is werd pas naar deel 2 gegaan waar deze werd opgesplitst in drie delen. Eerst werd de global memory volledig getest alvorens over te gaan naar shared memory en vervolgens constant memory. Als deze alle drie werkten werd dit in een loop gestoken en de data omgezet naar een csv file en genereerden we hieruit 3 grafieken.

Code

Deel 1

Na de afbeeldingen in te lezen en de groottes te bepalen van o.a. de image size, blocks en warps wordt de image array gesplits van RGBRGB naar RRGGBB. Dit wordt verwezenlijkt d.m.v. een offsetbepaling voor blauw en groen waarbij we telkens het 3e element nemen + de eventuele offset. Vervolgens converteren we de data via malloc om deze op de GPU te plaatsen. Dit proces gebeurt op dezelfde manier als de andere labo's waarbij we pointers aanmaken om een geheugenplaats te reserveren voor de data. Voor dit deel werden twee verschillende kernels vergeleken. 1 waarbij we rood, groen en blauw door elkaar hadden en 1 waarbij we de array van eerst rood, dan groen en vervolgens blauw gebruiken. Beide voeren dezelfde functie uit waarbij we rood inverteren. Uit resultaten blijkt hier dat de inverseImageRRGGBB sneller is dan de inverseImageRGBRGB. Het inverteren als RGBRGB formaat neemt $1093\mu s$ in beslag, terwijl het inverteren met als formaat RRGGBB dit $9\mu s$ in beslag neemt. Dit is natuurlijk wel zonder de duur van de bewerkingen die nodig zijn om van het RGBRGB naar het RRGGBB formaat te gaan.

Deel 2

Na het aanmaken van de matrices, deze te alloceren op de GPU en de grootte's te bepalen wordt de data naar de kernel omgezet. Hierin worden eerst i en j bepaald die het aantal rijen en kolommen voorstellen. Vervolgens worden de

limieten vastgesteld en voert men telkens de berekingen uit voor één plaats. Deze wordt dan gekoppeld met de matrix die het resultaat voorstelt.

Bij shared-memory gaan we ervoor zorgen dat we de matrices in de kernel in de GPU eerst in shared-memory omzetten alvorens we de bewerkingen erop gaan uitvoeren. Dit gebeurd met volgende commando's:

```
shared int shared_a[N][N];
shared int shared_b[N][N];
```

en

```
shared_a[threadIdx.y][threadIdx.x] = a[i * n + threadIdx.x];
shared_b[threadIdx.y][threadIdx.x] = b[(threadIdx.y * n) + j];.
```

Hierbij is het belangrijk dat we voor we de berekeningen uitvoeren we eerst synchroniseren waarbij anders data verloren zou kunnen gaan. Verder blijft de code hetzelfde als deze van global-memory.

Via constant memory worden de matrices a en b via het commando

```
cudaMemcpyToSymbol(d_a_constant, a_constant, N * N * sizeof(int));
```

gekoppeld met de GPU. Deze d_a_constant is nu geen pointer meer die een geheugenplaats vrij houdt, maar zijn nu plaatsen in de constant memory. Deze worden aangemaakt m.b.v.
`__constant__ int d_a_constant[N*N];`. Hierdoor heeft men enkel nog een pointer c nodig voor het resultaat in weg te steken van de vermenigvuldiging van de matrices.

Het verschil in berekentijd kunnen we zien in volgende grafieken:

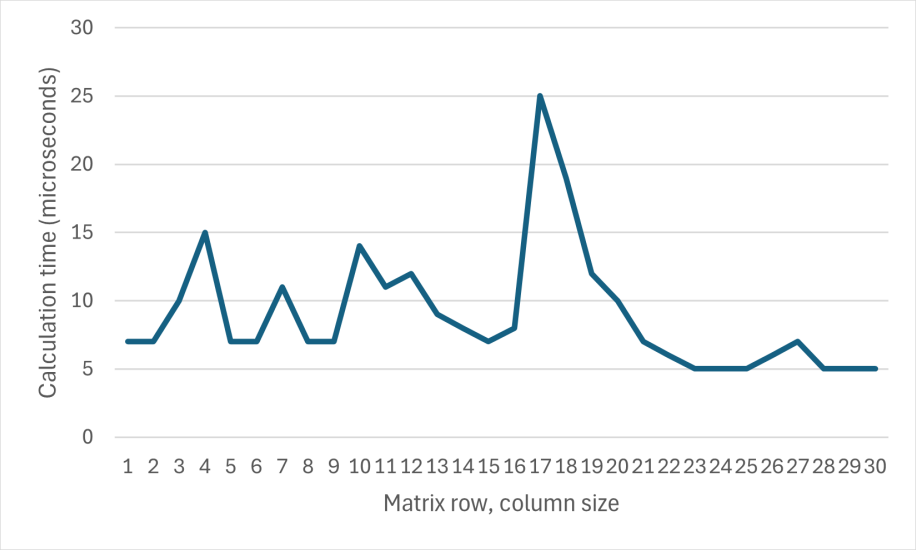


Figure 1: Global memory

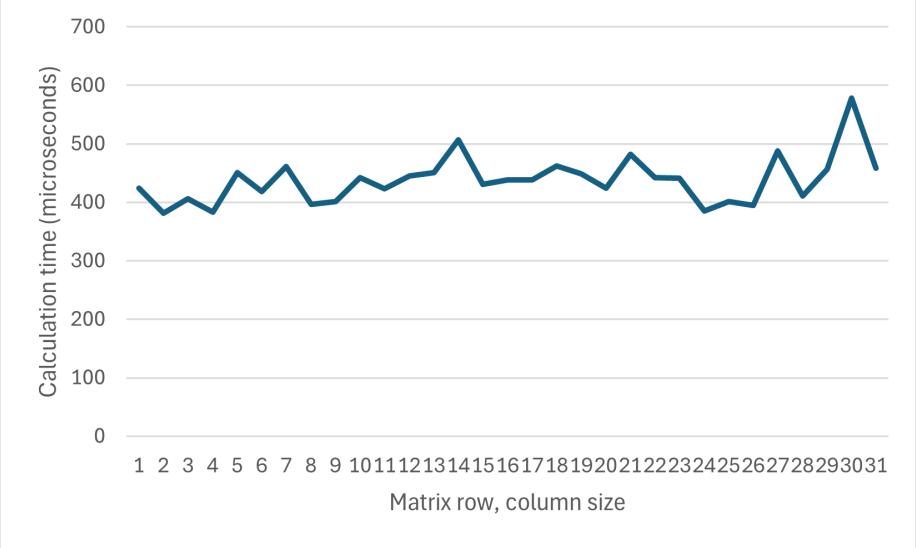


Figure 2: Shared memory

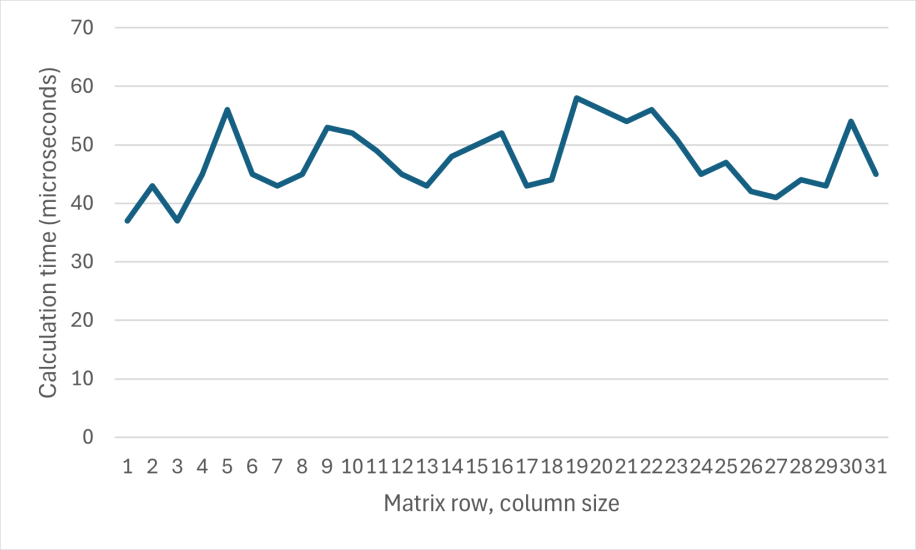


Figure 3: Constant memory

We zien dat global memory het snelst is, vervolgens komt constant memory en als laatste shared memory. Dit komt niet overeen met wat we eigenlijk verwachten. Als alles goed zou gaan, zou shared memory het snelst moeten zijn. Dit kan eventueel komen door een fout in de code. Dit zou mogelijks ook kunnen verklaren waarom er niet echt een verband aanwezig is tussen de grootte van de matrices en de berekentijd.